

BATERÍA DE PREGUNTAS - TEM-742

TECNOLOGÍAS EMERGENTES II

Desarrollo de Aplicaciones Empresariales

1. ¿Qué es una aplicación empresarial?

Una aplicación empresarial es un software diseñado para satisfacer las necesidades de una organización, facilitando procesos como ventas, gestión de recursos, finanzas, logística, atención al cliente, entre otros. Suelen ser complejas, integradas y escalables.

2. ¿Cuáles son las características clave de una aplicación empresarial moderna?

Las principales características son:

- **Escalabilidad:** Capacidad de crecer sin perder rendimiento.
- **Seguridad:** Protección de datos y accesos.
- **Integración:** Capacidad de comunicarse con otros sistemas.
- **Mantenibilidad:** Facilidad de actualizar y corregir errores.
- **Interoperabilidad:** Funcionamiento entre diferentes plataformas y tecnologías.

3. ¿En qué se diferencia una arquitectura monolítica de una basada en microservicios?

- **Monolítica:** Toda la lógica de negocio está unificada en una sola aplicación. Es más simple de desarrollar, pero difícil de escalar y mantener.
- **Microservicios:** Divide la aplicación en pequeños servicios independientes que se comunican entre sí. Facilita la escalabilidad y el mantenimiento.

4. ¿Qué ventajas ofrece la arquitectura de microservicios?

- Escalabilidad individual de componentes
- Despliegue y desarrollo independientes
- Mejor tolerancia a fallos
- Uso de diferentes lenguajes o tecnologías según el servicio
- Alineación con metodologías ágiles

5. ¿Qué es un CRM y qué función cumple en una empresa?

CRM (Customer Relationship Management) es un sistema que gestiona la relación con los clientes, facilitando la organización de contactos, seguimi

6. ¿Qué diferencia hay entre CRM, ERP y SRP?

- **CRM:** Gestión de relaciones con clientes.

- **ERP (Enterprise Resource Planning):** Integra y gestiona procesos empresariales como finanzas, producción, logística.
- **SRP (Supplier Relationship Planning):** Gestiona relaciones y procesos con proveedores.

7. ¿Por qué es importante la seguridad en las aplicaciones empresariales?

Porque estas aplicaciones manejan información sensible como datos de clientes, operaciones financieras y estrategias corporativas. Una brecha de seguridad puede causar pérdidas económicas, daño reputacional y sanciones legales.

8. ¿Qué herramientas se pueden utilizar en Python para desarrollar aplicaciones empresariales?

- **Flask:** Framework web ligero y flexible.
- **SQLAlchemy:** ORM para bases de datos.
- **Jinja2:** Motor de plantillas HTML.
- **Marshmallow:** Serialización de datos.
- **Gunicorn/Uvicorn:** Servidores WSGI/ASGI.
- **Docker:** Contenedores para despliegue.

9. ¿Qué ventajas ofrece Flask para el desarrollo empresarial?

- Minimalista y extensible
- Facilidad de integración con otras tecnologías
- Control total sobre la arquitectura
- Ideal para microservicios
- Gran comunidad y documentación

10. ¿Cómo se logra la interoperabilidad en aplicaciones empresariales?

A través de APIs (REST, GraphQL), formatos de intercambio de datos como JSON o XML, uso de estándares abiertos, y middleware que permite que sistemas distintos se comuniquen eficazmente.

Desarrollo Backend y persistencia de datos

1. ¿Cómo se define una ruta en Flask y qué función cumple?

Una ruta en Flask se define mediante un decorador como `@app.route('/ruta')`, que asocia una URL con una función específica. Esta función se ejecuta cuando se recibe una solicitud HTTP en esa ruta, permitiendo devolver una respuesta adecuada (HTML, JSON, etc.). ###

2. ¿Qué tipos de solicitudes HTTP se pueden manejar en Flask?

Flask permite manejar varios métodos HTTP, principalmente:

- **GET:** Obtener datos del servidor.
- **POST:** Enviar datos al servidor.
- **PUT/PATCH:** Actualizar recursos existentes.

- **DELETE:** Eliminar recursos. Se controlan con el parámetro `methods=["GET", "POST"]` en la definición de la ruta. ### 3. ¿Cómo se conecta Flask a una base de datos relacional como SQLite3 o MySQL?

Flask se conecta a una base de datos relacional como SQLite3 o MySQL utilizando controladores específicos y, generalmente, un ORM como SQLAlchemy para facilitar el manejo de datos.

Para **SQLite3**, se puede usar directamente con SQLAlchemy, ya que viene integrado:

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///mi_base.db'
```

Para **MySQL**, se necesita instalar un conector como `mysqlclient` o `pymysql`, y la URI de conexión sería algo como:

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+pymysql://  
usuario:contraseña@localhost/nombre_base'
```

Luego se inicializa SQLAlchemy:

```
from flask_sqlalchemy import SQLAlchemy  
db = SQLAlchemy(app)
```

Esto permite interactuar con la base de datos usando clases y métodos de Python, en lugar de escribir SQL puro.

4. ¿Qué diferencias hay entre bases de datos relacionales y no relacionales?

- **Relacionales:** Estructuran los datos en tablas con relaciones (MySQL, PostgreSQL). Son consistentes y basadas en SQL.
- **No relacionales (NoSQL):** Almacenan datos en documentos, grafos o claves-valor (MongoDB, Redis). Son más flexibles y escalables horizontalmente, ideales para datos no estructurados.

5. ¿Qué es un ORM y por qué se utiliza en el backend?

Un ORM (Object-Relational Mapper) es una herramienta que permite mapear objetos de un lenguaje de programación (como clases en Python) a tablas en una base de datos relacional.

Se utiliza en el backend porque simplifica el acceso a los datos, evitando la necesidad de escribir consultas SQL manuales. En su lugar, se manipulan objetos Python, lo que mejora la productividad, legibilidad y mantenibilidad del código.

Por ejemplo, con un ORM como SQLAlchemy, se puede crear, consultar o actualizar registros usando sentencias Python como:

```
nuevo_usuario = Usuario(nombre='Bruno')  
db.session.add(nuevo_usuario)  
db.session.commit()
```

Esto reemplaza al SQL tradicional y permite trabajar de forma más segura y estructurada.

6. ¿Cómo se define un modelo de datos en SQLAlchemy?

Un modelo se define como una clase Python que hereda de `db.Model`. Los atributos de la clase representan las columnas de una tabla. Por ejemplo:

```
class Usuario(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    nombre = db.Column(db.String(100))
```

7. ¿Cómo se optimiza una consulta a la base de datos?

- Utilizando índices en campos frecuentemente consultados.
- Filtrando los datos con `query.filter()` para evitar traer todo el conjunto.
- Usando `join` para evitar múltiples consultas separadas.
- Limitando los resultados (`.limit(n)`).
- Evitando cargas innecesarias (`lazy='dynamic'` o `selectinload`).

8. ¿Qué es una transacción y cómo se maneja en Flask con SQLAlchemy?

Una transacción agrupa un conjunto de operaciones que deben completarse exitosamente o deshacerse en conjunto. En SQLAlchemy, se puede usar:

```
try:
    db.session.add(obj)
    db.session.commit()
except:
    db.session.rollback()
```

Esto asegura la integridad de los datos ante errores.

Seguridad en Aplicaciones Empresariales

1. ¿Cuál es la diferencia entre autenticación y autorización?

La **autenticación** verifica la identidad del usuario (por ejemplo, con usuario y contraseña), mientras que la **autorización** determina si ese usuario tiene permisos para acceder o ejecutar ciertas acciones dentro del sistema. ### 5. ¿Por qué es importante cifrar los datos sensibles?

Porque protege información crítica como contraseñas, números de tarjeta o datos personales, incluso si la base de datos es comprometida. El cifrado asegura que los datos sean ilegibles sin la clave adecuada.

6. ¿Qué algoritmos se usan comúnmente para cifrado de contraseñas?

Para contraseñas se usan algoritmos de **hash seguro con sal** como `bcrypt`, `argon2` o `PBKDF2`. No se deben almacenar contraseñas en texto plano. Estos algoritmos impiden que un atacante pueda obtener la contraseña original fácilmente.

7. ¿Por qué es fundamental usar HTTPS en una aplicación empresarial?

Porque HTTPS cifra la comunicación entre el cliente y el servidor usando certificados **SSL/TLS**, lo que impide que los datos sean interceptados o modificados por terceros durante la transmisión.

8. ¿Qué técnicas se aplican para validar y sanitizar las entradas del usuario?

- **Validar:** comprobar que el dato cumple un formato o regla esperada (por ejemplo, un email válido).
- **Sanitizar:** eliminar o neutralizar caracteres peligrosos (por ejemplo, scripts, comillas, etiquetas HTML) para evitar ataques.

9. ¿Qué herramientas se pueden usar en Flask para mejorar la seguridad?

- **Flask-Login:** para autenticación y sesiones seguras.
- **Flask-WTF:** para formularios con protección CSRF integrada.
- **Flask-Talisman:** para forzar HTTPS y aplicar políticas de seguridad.
- **bcrypt / passlib:** para cifrado de contraseñas.

10. ¿En qué consiste la auditoría de actividades y por qué es importante el logging?

La auditoría permite registrar eventos importantes del sistema (inicio de sesión, errores, cambios críticos). El logging ayuda a detectar actividades sospechosas, investigar incidentes de seguridad y cumplir con normativas. En Flask se puede integrar fácilmente con **logging** de Python o servicios externos como Sentry.

Web Services y APIs

1. ¿Qué es una API y para qué se utiliza en el desarrollo web?

Una API (Application Programming Interface) es un conjunto de reglas que permite que diferentes aplicaciones se comuniquen entre sí. En el desarrollo web, se utiliza para exponer funcionalidades o datos de un sistema para que otros sistemas los consuman. ### 2. ¿Qué es una API RESTful?

Una API RESTful sigue el estilo de arquitectura REST, que se basa en el uso de métodos HTTP (GET, POST, PUT, DELETE) y recursos identificables por URLs. Es una forma simple, escalable y estandarizada de diseñar APIs en la web.

3. ¿Cómo se diseña una API RESTful básica con Flask?

En Flask se definen rutas para los diferentes endpoints y se manejan las solicitudes HTTP. Por ejemplo:

```
@app.route('/usuarios', methods=['GET'])
def obtener_usuarios():
    return jsonify(lista_usuarios)
```

Se usa `jsonify()` para devolver respuestas en formato JSON y se pueden manejar errores, filtros y paginación.

4. ¿Cómo se puede consumir una API externa desde una aplicación Flask?

Se puede usar la librería `requests` para enviar solicitudes HTTP a una API externa:

```
import requests
respuesta = requests.get('https://api.ejemplo.com/datos')
datos = respuesta.json()
```

Esto permite integrar información de terceros, como servicios de clima, pagos o mapas.

5. ¿Qué mecanismos se usan para proteger una API con autenticación y autorización?

Los mecanismos comunes son:

- **Tokens (JWT):** Se usa un token firmado que el cliente debe enviar en cada solicitud.
- **API Keys:** Claves únicas para cada cliente autorizado.
- **OAuth2:** Protocolo de autorización más robusto usado por plataformas como Google o Facebook.

6. ¿Qué es un JWT y cómo funciona?

JWT (JSON Web Token) es un token codificado que contiene información del usuario y su autorización. Es enviado en la cabecera `Authorization` y permite validar al usuario sin mantener una sesión en el servidor.

7. ¿Por qué es importante documentar una API?

Porque facilita que otros desarrolladores puedan entender cómo usarla correctamente. Una buena documentación incluye rutas, métodos, parámetros, ejemplos de solicitudes/respuestas y códigos de estado HTTP. Mejora la colaboración y reduce errores.

8. ¿Qué herramientas se pueden usar para documentar APIs en Flask?

- **Swagger / OpenAPI:** Estándar para documentación interactiva.

Despliegue y monitoreo

1. ¿Qué es un servidor web y cuál es su función en el despliegue de una aplicación?

Un servidor web es un software que gestiona las solicitudes HTTP y entrega contenido web a los clientes. En el despliegue de aplicaciones, permite servir archivos estáticos y manejar las peticiones hacia la aplicación backend (por ejemplo, usando Nginx o Apache).

2. ¿Qué es Render y por qué es una buena opción para desplegar aplicaciones Flask?

Render es una plataforma de alojamiento en la nube que permite desplegar aplicaciones web, APIs, bases de datos y servicios estáticos de forma sencilla. Es ideal para proyectos académicos o personales porque ofrece un flujo de despliegue automatizado desde GitHub y un plan gratuito con recursos suficientes para pequeñas apps.

3. ¿Cuáles son los pasos básicos para desplegar una aplicación Flask en Render?

1. Subir el proyecto a un repositorio en GitHub.
2. Crear un archivo `requirements.txt` con las dependencias.
3. Añadir un archivo `Procfile` que indique cómo ejecutar la app.
4. Conectarse a GitHub desde Render y seleccionar el repositorio.
5. Configurar las variables de entorno necesarias y desplegar.

4. ¿Qué debe contener el archivo `Procfile` para una aplicación Flask en Render?

Debe indicar el comando para ejecutar el servidor. Por ejemplo:

```
web: gunicorn app:app
```

Esto significa que se usará Gunicorn para ejecutar el objeto `app` del archivo `app.py`.

5. ¿Qué ventajas tiene usar Render frente a otras plataformas como Heroku?

- Despliegue automático desde GitHub.
- Plan gratuito generoso sin necesidad de tarjeta de crédito.
- Soporte para HTTPS y dominios personalizados incluidos.
- Panel simple e intuitivo.
- Soporte para tareas cron y servicios de backend permanentes.

Git y GitHub

1. ¿Qué es Git y para qué se utiliza?

Git es un sistema de control de versiones distribuido que permite registrar los cambios en el código fuente de un proyecto a lo largo del tiempo. Facilita el trabajo colaborativo y el seguimiento de versiones, ramas y actualizaciones.

2. ¿Cuál es la diferencia entre Git y GitHub?

Git es la herramienta de control de versiones que se usa localmente en tu máquina, mientras que GitHub es una plataforma en línea que permite alojar repositorios Git, colaborar con otros, hacer revisiones de código y gestionar proyectos.

3. ¿Qué comando se utiliza para inicializar un repositorio Git?

```
git init
```

Este comando crea un nuevo repositorio Git en el directorio actual, permitiendo comenzar a controlar versiones del proyecto.

4. ¿Cómo se sube un proyecto local a un repositorio en GitHub?

1. Crear un repositorio en GitHub.
2. Conectar el repositorio local al remoto: `bash git remote add origin https://github.com/usuario/repositorio.git`
3. Subir el código: `bash git add . git commit -m "Primer commit" git push -u origin main ###`

5. ¿Qué hace el comando `git clone`?
`git clone` copia un repositorio remoto (por ejemplo, de GitHub) a tu computadora local, incluyendo todo su historial y archivos:

```
git clone https://github.com/usuario/repositorio.git
```

6. ¿Qué diferencia hay entre `git pull` y `git push`?

- `git pull` descarga y fusiona los cambios desde el repositorio remoto al local.
- `git push` sube los cambios desde el repositorio local al remoto (GitHub).

7. ¿Qué es un commit en Git?

Un **commit** es un registro de cambios guardado en el repositorio. Cada commit tiene un mensaje que describe lo que se ha modificado. Es como un “punto de guardado” en el historial del proyecto.

8. ¿Qué es una rama (branch) y para qué se utiliza?

Una **rama** es una línea paralela de desarrollo dentro del mismo proyecto. Se utiliza para trabajar en nuevas características o correcciones sin afectar el código principal. La rama principal suele llamarse `main` o `master`.

9. ¿Qué archivos se deben ignorar con `.gitignore`?

En `.gitignore` se listan archivos o carpetas que no deben incluirse en el repositorio (como archivos temporales, configuraciones locales, entornos virtuales o dependencias). Por ejemplo:

```
__pycache__/  
.env  
venv/  
*.pyc
```

10. ¿Qué es un pull request (PR) en GitHub?

Un **pull request** es una solicitud para fusionar cambios de una rama (generalmente de un colaborador) con la rama principal del repositorio. Permite revisar, comentar y aprobar los cambios antes de integrarlos.